

Defense Question Bank

GCTU Campus Navigator · questions a supervisor would put to you, with a model answer under each. Answers are in the first person, as you would say them to the panel.

Your 30-second opening pitch (learn this by heart)

"The GCTU Campus Navigator is a Progressive Web App that helps students and visitors find buildings on the Tesano campus. It computes the shortest walking route with Dijkstra's algorithm, gives turn-by-turn directions, tracks you live with GPS, and keeps working offline. It also has a help desk where students submit support requests, which are stored in MongoDB and answered by an automatic keyword bot and by admins who sign in securely with JWT. The frontend is React with Leaflet, the backend is Node.js with Express, and the database is MongoDB Atlas."

A The project in general

Q1. What problem does your system solve?

New students, visitors and even staff struggle to find buildings, lecture halls and offices on campus. Signs are limited, and general tools like Google Maps don't know internal campus paths or places like "COLT" or "Room G6". My app gives campus-specific, step-by-step walking directions, works without internet, and adds a help desk so students can get answers to common questions and file support requests in one place.

Q2. Who are the users, and what can each of them do?

- **Students and visitors:** explore buildings, get directions, use live GPS or the walk demo, read FAQs, call hotlines and submit help requests. No account is needed.
- **Admins (support staff):** sign in, view all help-desk tickets and mark them as replied.

Q3. What are the main features?

1. An interactive campus map (Leaflet) with building markers and light and dark themes.
2. Shortest-path routing (Dijkstra) with turn-by-turn directions.
3. Live GPS tracking and a walking simulation for demos.
4. Offline support through a service worker (PWA, installable to the home screen).
5. A help desk: FAQs, hotlines, ticket submission and an automatic bot reply.
6. An admin login (bcrypt and JWT) for managing tickets.

Q4. Why build your own app instead of using Google Maps?

Google Maps knows public roads, not the internal walkways between our blocks, and it doesn't know campus names like COLT, SGSR or Great Hall. Its terms also don't allow caching map tiles for offline use, and its API needs a billing account. My app uses the campus's own path network, works offline, and adds campus services like the help desk.

Q5. Which development methodology did you follow?

I built the system incrementally: first the map and routing, then GPS, then the backend and database, then the help desk and admin features. After each stage I tested and refined, which is close to an iterative, Agile-style approach.

Tip: make this answer match the methodology you named in Chapter 3 of your report.

B Architecture and design decisions

Q6. Describe the architecture of your system.

It is a three-tier client-server architecture:

- **Presentation tier:** a React single-page app in the browser (the map, routing, GPS and UI).
- **Application tier:** a Node.js and Express REST API (validation, authentication, the ticket logic).
- **Data tier:** MongoDB Atlas, reached through Mongoose.

They communicate over HTTP using JSON. In development, Vite runs the frontend on port 3000 and forwards `/api` requests to Express on port 5000.

Q7. Where is the shortest path calculated, and why there?

On the user's device, in the browser. The campus graph is tiny (15 nodes), so computing locally is instant, costs the server nothing, and most importantly **still works with no internet**, which matters for a navigation app. The server is used for what has to be shared or stored permanently: tickets, admin accounts, FAQs and contacts.

Q8. So what is the database used for if routing is done in the frontend?

It stores everything that must persist or be shared between users: **tickets** submitted by students, **admin accounts** with hashed passwords, and the **FAQs and contacts**. It also holds the **buildings and walkway graph**, which are available through `/api/buildings`, `/api/buildings/search` and `/api/graph`. That data is seeded from the same file the app bundles, so the two copies are always identical.

Q9. Why React, Express and MongoDB (a MERN-style stack)?

- **React:** its component model fits an app with three independent tabs, and it has a mature ecosystem, including react-leaflet for maps.
- **Express:** lightweight and simple for a REST API, and it's JavaScript on both sides, so there is one language throughout.
- **MongoDB:** our data is document-shaped (a building contains lists of rooms, aliases and facts) and it has built-in text search.

Q10. What is a REST API, and how is yours RESTful?

REST organises an API around **resources** with URLs and uses HTTP methods for actions. My resources are buildings, the graph, FAQs, contacts and tickets. GET reads (`GET /api/tickets`), POST creates (`POST /api/tickets`) and PATCH partially updates (`PATCH /api/tickets/:ticketId`). It is stateless: each request carries everything needed, including the JWT, and the server keeps no session. Status codes carry meaning: 200, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found and 500.

C Frontend

Q11. How is the frontend structured?

`main.jsx` renders `App`, which shows three tabs: `CampusHome`, `NavigationPanel` and `HelpDesk`. I separated the code by responsibility:

- **components/**: UI only, split into small reusable pieces (e.g. `navigation/CampusMap`, `helpdesk/TicketForm`, a shared `ui/Banner`).
- **hooks/**: reusable stateful logic: `useLiveGps`, `useWalkSimulation`, `useAdminSession`, `useTheme`.
- **lib/**: pure functions with no React, e.g. `routing.js` (Dijkstra) and `geo.js` (Haversine), so they can be tested on their own.
- **data/**: the campus graph and help-desk content, shared with the database seed.

Q12. How do the components communicate? For example, "Get Directions" on the Campus tab.

Through props and lifting state up. `CampusHome` receives an `onNavigateTo` callback. Clicking a card calls it with the building name, and `App` stores that as `presetDestination` and switches to the `Navigate` tab. `NavigationPanel` receives the preset, finds the building, sets it as the destination with the Main Gate as the start, then clears the preset.

Q13. Which React hooks did you use, and for what?

- `useState`: UI state (the selected start and destination, the route, GPS state, form fields).
- `useEffect`: side effects, such as recomputing the route when the start or destination changes, starting and cleaning up GPS watchers and timers, loading data from the API, and registering the service worker.
- `useRef`: values that must survive renders without triggering one, such as the GPS watch ID, the simulation interval, and the current route for the GPS callback.
- `useMemo`: the route is *derived* from the chosen start and destination, not copied into extra state.
- **Custom hooks** I wrote: `useLiveGps` and `useWalkSimulation` hold all the tracking logic, so the map component only renders.

Q14. When you switch tabs, is the map destroyed and recreated?

No. All three tabs stay mounted, and switching only toggles a CSS class. The Leaflet map isn't torn down, so the route, zoom level and any running GPS session survive a tab change. When the tab becomes visible again I call `invalidateSize()` so Leaflet re-measures its container.

Q15. How does the frontend talk to the backend?

With the browser's `fetch()` API, sending and receiving JSON. Every response is checked: if it isn't a successful list, it is treated as an error rather than stored, and the Help Desk falls back to a bundled copy of the FAQs and contacts. Admin requests add an `Authorization: Bearer <token>` header.

Q16. How does dark mode work?

The preference is stored in `localStorage`, defaulting to the operating system's setting via `prefers-color-scheme`. It adds a `dark` class to `<body>`, which switches the CSS variables, and, because OpenStreetMap has no dark style, a CSS filter (`invert` plus `hue-rotate`) darkens the map tiles while the markers and route keep their true colours.

D Mapping and algorithms (expect the most questions here)

Q17. How is the map displayed?

With **Leaflet**, an open-source JavaScript map library, through **react-leaflet**. The background comes from **OpenStreetMap tiles**: 256×256-pixel images fetched by zoom level and x/y position. They are free and need no API key. I first used CARTO's tiles, but its free tier started stamping "API KEY REQUIRED" across every tile, so I switched. On top I draw my own layers: building markers, start and destination pins, the route polyline, a distance badge, and the GPS marker with its accuracy circle. The map starts at the campus centre [5.5966, -0.2234] at zoom 18.

Q18. How did you represent the campus for routing?

As a **weighted undirected graph**. There are **15 nodes**: 8 buildings, the Main Gate and 6 walkway junctions where paths meet. (The Library and the Florence Onny Auditorium are inside buildings, not separate nodes; see Q61.) They are connected by **26 edges**, the walkable paths. Each edge's weight is the real distance in metres between its endpoints. It is undirected because every path can be walked both ways.

Q19. Where did the coordinates come from?

I placed each building and junction by reading its latitude and longitude off Google Maps satellite imagery, then corrected them after testing, because early estimates produced wrong turn directions. The campus boundary comes from OpenStreetMap. The graph is only as accurate as those measurements, which I state as a limitation.

Q20. How do you calculate the distance between two coordinates?

With the **Haversine formula**, which gives the great-circle distance on a sphere. Simple Pythagoras on latitude and longitude would be wrong, because degrees aren't equal distances.

$$a = \sin^2(\Delta\phi/2) + \cos \phi_1 \cdot \cos \phi_2 \cdot \sin^2(\Delta\lambda/2) \quad d = 2R \cdot \text{atan2}(\sqrt{a}, \sqrt{1-a}), \quad R = 6,371,000 \text{ m}$$

Q21. Explain how Dijkstra's algorithm works in your system.

1. Set every node's distance to infinity and the start's to 0. All nodes start unvisited.
2. Pick the unvisited node with the smallest distance. If it's the destination, stop.
3. For each neighbour, compute *current distance* + *edge length*. If that is shorter than what the neighbour has, update it and record `previous[neighbour] = current`.
4. Mark the current node visited and repeat.
5. Rebuild the path by following `previous` back from the destination.

Example: Main Gate → Main Gate walkway (61.8 m) → Graduate Block walkway (32.4 m) → Engineering (16.7 m) → Cafeteria (19.3 m) → FoCIS (93.6 m) = **223.8 m**.

Q22. Why Dijkstra and not A* or BFS?

BFS finds the path with the fewest steps, not the shortest distance, and my edges have different lengths. **A*** is Dijkstra plus a heuristic that pays off on huge graphs like city road networks. With 15 nodes, Dijkstra answers instantly and is guaranteed optimal because all weights (distances) are non-negative. If the graph grew, switching to A* would be easy: straight-line Haversine distance is an admissible heuristic.

Q23. What is the time complexity of your implementation?

I find the minimum with a linear scan and check the edge list for each visited node, so it is $O(V \cdot (V + E))$. With $V = 15$ and $E = 26$ that is only a few hundred operations, well under a millisecond. With a binary-heap priority queue and an adjacency list it would be $O((V + E) \log V)$, which only matters for large graphs.

Q24. How are the turn-by-turn directions generated?

At each node on the path I compute the **bearing** of the incoming segment and of the outgoing segment using `atan2`, and take the difference, normalised to -180° to $+180^\circ$. Under 22° means "continue straight", $22-65^\circ$ "bear slightly", $65-125^\circ$ "turn", and over 125° "sharp turn". A positive angle is a right turn and a negative one is left. The first step is always "Depart from ...", and each step shows its distance.

Q25. How does live GPS tracking work?

I use the browser's **Geolocation API**, `navigator.geolocation.watchPosition()` with high accuracy enabled, which calls my code on every new position fix. Each fix moves the blue marker and draws an accuracy circle. I then **project the position onto the route**: treating the small campus area as a flat metre grid, I find the closest point on any route segment. That gives the distance off route, the distance travelled, the distance remaining, and which step to highlight. If the user is more than 300 m away, the app says they aren't on campus. Within **20 m of the destination** (using only fixes accurate to ± 50 m, so one bad reading can't trigger it) it shows "You have arrived" and announces it by voice, once per route. While navigating it also keeps the screen on (Screen Wake Lock API) so the phone doesn't lock and pause tracking. GPS requires HTTPS or localhost, and the app checks for that.

Q26. What if the GPS signal is poor or the user denies permission?

A permission denial stops tracking and shows a message. Timeouts and "position unavailable" are usually temporary, so tracking continues and `watchPosition` keeps retrying. The accuracy circle shows the user how reliable the current fix is, and snapping to the route stops small GPS errors from breaking the guidance.

Q27. What is the Walk Demo?

A simulation for demonstrating away from campus. A virtual GPS marker walks the route at 1.4 m/s, average walking speed, sped up $5\times$ for the demo. It updates every 100 ms and interpolates the position along each segment, so the motion is smooth. The directions advance as it goes, and on arrival the browser announces the destination with speech synthesis, including the indoor step for places like the Library.

Why it works on phones too: iPhones only allow speech that was first started inside a tap, and the arrival fires later from a timer. So the app says "Starting navigation to..." inside the button tap, which unlocks speech for the rest of the walk. On Android it clears any stuck announcement and sets an English voice first.

E Backend and API

Q28. Walk us through what happens when a request reaches your server.

In `server/index.js`, each request passes through `cors()`, then `express.json()` to parse the body, then the matching router (`campus`, `helpdesk`, `tickets` or `admin`). Protected routes run the `authenticateAdmin` middleware first. The handler validates input, queries MongoDB through Mongoose and returns JSON. Unknown routes get a JSON 404 and errors get JSON 400 or 500 responses. The server only starts listening once the database connection succeeds.

Q29. List your API endpoints.

Endpoint	Purpose	Access
GET <code>/api/health</code>	Server liveness check	public
GET <code>/api/buildings</code> , <code>/buildings/:id</code> , <code>/buildings/search?q=</code>	Buildings, one building, full-text search	public
GET <code>/api/graph</code>	Walkway graph (nodes and edges)	public
GET <code>/api/faqs</code> , <code>/faqs/search?q=</code> , <code>/api/contacts</code>	Help-desk content	public
POST <code>/api/tickets</code>	Submit a help request	public
POST <code>/api/admin/login</code>	Get a JWT	public
GET <code>/api/tickets</code> , PATCH <code>/api/tickets/:ticketId</code>	List and update tickets	admin JWT

Q30. What is middleware? Give an example from your code.

Middleware is a function that runs between receiving a request and sending the response. It can read or modify the request, stop it, or pass it on with `next()`. Examples: `express.json()` parses bodies, and my own `authenticateAdmin` reads the Bearer token, verifies it with `jwt.verify`, and either calls `next()` or returns 401.

Q31. Explain the full life of a help-desk ticket.

1. The student fills in the form. HTML5 `required` attributes and a JavaScript check block empty fields.
2. The frontend sends `POST /api/tickets`. The server checks that name, subject and message exist (otherwise 400).
3. The server generates an ID like `TKT-4821`, retrying if it already exists, saves the ticket with status "open", and returns **201 Created**.
4. 2.5 seconds later the bot scans the text for keywords (Wi-Fi, clinic, portal, fees), writes a matching reply and sets status "replied". It only touches tickets that are still open, so it never overwrites an admin's reply.
5. An admin signs in, sees the ticket and can mark it replied.

Q32. Is the bot artificial intelligence?

No. It is a rule-based keyword matcher. I'd call it an automated responder, not AI. It answers the most common categories instantly. Replacing it with a natural-language model is listed as future work.

Q33. How do you handle errors in the API?

Every route handler goes through a shared `asyncHandler` wrapper, so any error reaches one central error handler instead of each route repeating its own `try/catch`. Errors come back as JSON with a proper status code: 400 for invalid input, 401 for authentication failures, 404 for things that don't exist, and 500 for unexpected failures. A global error handler turns malformed JSON into a 400. The bot's delayed database update has its own `try/catch`, so a database hiccup there can't crash the server.

Q34. What does CORS mean, and why do you use it?

Cross-Origin Resource Sharing. Browsers block a page on one origin from calling an API on another unless the API allows it. The `cors()` middleware adds the headers that allow it, so the frontend can call the API even when they are hosted separately. In production I would restrict it to the app's own domain.

F Database

Q35. Which database did you use, and why NoSQL rather than SQL?

MongoDB Atlas, a cloud-hosted document database. My data is naturally document-shaped: a building holds arrays of aliases, facts and rooms, which in SQL would need several tables and joins. MongoDB also provides built-in **text indexes** for search, a flexible schema while the project evolved, and free managed hosting with backups.

Q36. What collections do you have?

Seven: **buildings**, **graphnodes**, **graphedges**, **faqs**, **contacts**, **tickets** and **admins**. Every document gets automatic `createdAt` and `updatedAt` timestamps.

Q37. What is Mongoose, and why use it?

Mongoose is an ODM (Object-Document Mapper) for MongoDB in Node.js. It lets me define **schemas**: field types, required fields, defaults, allowed values (a ticket's status must be "open" or "replied") and unique fields such as `ticketId` and `username`. That gives structure and validation on top of MongoDB's flexible documents.

Q38. What indexes did you create, and why?

- **Unique indexes** on `buildings.id`, `tickets.ticketId`, `faqs.faqId` and `admins.username` to prevent duplicates.
- **Text indexes** on building name, short name, aliases and room names, and on FAQ question, answer and category. They power ranked full-text search, so `?q=COLT` or `?q=Room 66` finds the right building.

Q39. How did the initial data get into the database?

With a seed script, `npm run seed`. It clears the collections and inserts the buildings, graph, FAQs and contacts from `src/data/buildings.js` and `src/data/helpdesk.js`, the same files the frontend bundles. There is a single source of truth, so the offline app and the database can't drift apart.

Q40. Your database is in the cloud. What if the network blocks it?

I ran into this during development. Atlas's `mongodb+srv` address needs DNS lookups (SRV and TXT records) that phone hotspots often block. I handled it in two ways: `db.js` detects the failure and connects with a direct list of the cluster's hosts, retrying through public DNS if needed, and the configuration can use Atlas's standard host-list address, which needs no special lookups at all.

G Security

Q41. How are admin passwords stored?

Never in plain text. They are hashed with **bcrypt** using 10 salt rounds. bcrypt is deliberately slow and salts every hash, so identical passwords produce different hashes and brute-force or rainbow-table attacks are impractical. At login, `bcrypt.compare()` checks the typed password against the stored hash. One of my integration tests reads the database directly to confirm the stored value is a 60-character bcrypt hash.

Q42. What is a JWT, and how does your authentication work?

A JSON Web Token is a signed token in three parts: *header.payload.signature*. After a successful login the server signs one containing the admin's id and username with a secret key, valid for **8 hours**. The browser stores it and sends it as `Authorization: Bearer <token>`. The middleware verifies the signature and expiry on every protected request. It is stateless: nothing is stored on the server. Missing, forged or expired tokens get 401, and I have tests for all three.

Q43. Couldn't someone bypass the admin panel in the browser and read the tickets?

No. Hiding the admin panel in the UI is only for convenience. The real protection is on the server: `GET /api/tickets` and `PATCH /api/tickets/:id` refuse any request without a valid token signed with the server's secret, whatever the browser shows. Ticket data contains students' names and messages, so listing it is admin-only.

Q44. How do you protect against injection attacks?

Mongoose casts inputs to the types defined in each schema, and the login route rejects any username or password that isn't a plain string. So a NoSQL-injection payload such as `{"$ne": null}` gets a 400 instead of reaching the query. React escapes all text it renders, which protects against XSS. Wrong logins always return the same "Invalid credentials" message, so an attacker can't find out which usernames exist.

Q45. Where are secrets kept?

In a `.env` file (database URI, JWT secret, admin credentials) loaded with `dotenv`. It is listed in `.gitignore`, so it never goes into the repository, and a `.env.example` documents the required variables without real values.

Q46. What security weaknesses remain?

Being honest about these scores well with a panel:

- The JWT is stored in `localStorage`, so an XSS bug could steal it. An `httpOnly` cookie would be safer.
- There is no rate limiting on login yet (e.g. `express-rate-limit`).
- CORS is open to all origins in development.
- The system must run over HTTPS in production.

All four are straightforward to add.

H Testing

Q47. How did you test the system?

With automated tests in **Vitest: 49 UI tests** using React Testing Library in a simulated browser (jsdom), and **14 integration tests** that start the real Express server and check the results directly in MongoDB Atlas, with nothing mocked. I also tested manually on a phone for GPS and offline behaviour.

ID	What it proves
UT-01	The map renders at the campus centre with tiles and markers
UT-02	Route distance is computed correctly (Haversine and Dijkstra)
UT-03 / UT-04	A complete ticket is submitted; an incomplete one is blocked
UT-05	Admin login, token storage, sign-out, expired session
IT-01	Changing start or destination redraws the route
IT-02	A ticket POSTed to the API is really saved in MongoDB
IT-03 / IT- 04	Offline: the route stays, cached pages load, tiles fall back
Hooks	Walk demo moves along the route and arrives; live GPS snaps to the route, detects arrival (ignoring inaccurate fixes), handles denied permission and insecure pages
Speech	Announcements are spoken inside the tap and on arrival, in English, and survive a stuck queue or missing voice list
IT-05	bcrypt storage, JWT issuing, rejection of forged or expired tokens

Q48. What is the difference between a unit test and an integration test?

A **unit test** checks one piece in isolation, for example the distance function, or the HelpDesk form with the network faked. An **integration test** checks that several parts work together for real, for example the form's payload going through Express into MongoDB and being read back out.

Q49. Did you find bugs through testing?

Yes. For example, a thorough review before the defense found:

- The service worker failed to install in the production build because it tried to cache development-only files.
- The seed script silently did nothing on Windows.
- The ticket list was publicly readable.
- An error response from the API could crash the Help Desk page.

Each was fixed and a regression test was added.

I PWA, offline mode and deployment

Q50. What makes it a Progressive Web App?

It runs in any modern browser but behaves like an installed app. It can be added to the home screen (I capture the `beforeinstallprompt` event to show an Install banner), and a **service worker** makes it work offline. One codebase serves Android, iPhone and desktop, with no app-store installation.

Q51. How exactly does offline mode work?

The service worker (`public/sw.js`) intercepts network requests and uses a different strategy for each type:

- **App shell** (HTML, JS, CSS): served from the cache first, then refreshed in the background.
- **Map tiles**: cached on first view, and replaced by a "Map Offline" placeholder if missing.
- **API data**: network first so tickets are fresh, falling back to the cached copy when offline. Admin responses are never cached.

Routing needs no network at all, because the graph is bundled with the app.

Q52. How would you deploy this for the whole university?

It is ready for this: `npm run build` produces optimised static files, and `npm start` runs one Express server that serves both the app and the API on a single port. That can run on a university server or a Node host such as Render or Railway, with the database staying on MongoDB Atlas. Libraries (React, Leaflet) are split into separately cached files, so updates only re-download the small app code. The whole site must use **HTTPS**, because browsers only allow GPS and service workers on secure origins.

J Tough or tricky questions

Q53. You have a building search endpoint. Where is search in the app?

The endpoint (`/api/buildings/search`) is implemented and tested with a text index, but the current interface uses dropdowns and building cards instead. With ten buildings, a dropdown is faster than typing. The endpoint is ready for a search bar, or for other clients, once rooms and more locations are added.

Tip: your `docs/diagrams.md` shows a search flow from the frontend. Be ready for this question, or correct the diagram before you go in.

Q54. How does a student see the reply to their ticket?

Currently the reply is recorded in the database and shown on the admin dashboard, and the student gets a "Request filed" confirmation. Letting students look up their own ticket, for example by ticket ID, or notifying them by email is the next step I would build.

Tip: this is a genuine gap. Say it confidently as future work rather than getting defensive.

Q55. If a path is closed for construction, can your system handle it?

Not dynamically yet. The graph is static data, so I would remove or disable that edge and reseed. Because Dijkstra only considers existing edges, the next route calculation would automatically go around the closure. A future admin screen for editing the graph would make this possible without touching code.

Q56. Does "shortest route" mean fastest? What about stairs or accessibility?

It is shortest by distance. Walking speed is roughly constant on campus, so it is effectively the fastest too. The model doesn't yet include stairs, slopes or wheelchair access. Adding an attribute to each edge and a "step-free" mode that skips those edges is a clear extension.

Q57. Does it work inside buildings?

Partly. It guides you to the right building entrance, and for places inside buildings it adds the indoor step, for example the Library on the Admin block's first floor. Full room-by-room guidance needs floor plans, and GPS is unreliable indoors, so positioning there would use QR codes at entrances or Wi-Fi/Bluetooth beacons. See Q61–Q64.

Q58. Can your system scale if every student uses it at once?

Yes, largely by design. Routing runs on each phone, so thousands of users put no routing load on the server. The API mostly serves small read-only lists, and the app shell and tiles are cached on the device. Express can be run as several instances behind a load balancer, and Atlas scales the database separately.

Q59. What was the most difficult part, and how did you overcome it?

Accurate navigation. Estimated coordinates gave wrong turn directions, so I re-measured the nodes and added missing walkway edges, such as the direct FoCIS to Block C path. Making the GPS marker move visibly was another challenge: the map was re-centring on every fix, so the marker looked frozen. I changed it to pan only when the marker nears the edge of the view. Connecting to Atlas over a mobile hotspot that blocked DNS lookups was the main backend challenge (see Q40).

Tip: use your own words and real experiences here. Panels value honest reflection.

Q60. What are the limitations, and what would you do next?

• **Limitations:**

- Coordinates were placed by hand.
- Indoor guidance stops at the building entrance and floor (no room-by-room routing yet), and there is no step-free routing.
- The bot is keyword-based.
- Students can't yet track their own ticket.
- The security hardening listed in Q46 is still to do.

• **Next steps:**

- A search bar using the existing endpoint.
- Ticket lookup and notifications.
- An admin editor for the map graph.
- Step-free routes.
- Indoor floor plans.
- Deployment on the university's HTTPS domain.

K Indoor navigation and the change of scope

Q61. Your proposal included indoor navigation. What happened?

Indoor room-level navigation depended on floor plans from the university, which did not arrive in time. Rather than present nothing, I reprioritised: I delivered the outdoor layer first, which every indoor journey needs anyway (you must reach the building before you can find a room in it), and designed the system so indoor can be added without rework.

Tip: check how Chapter 1 of your report words this objective, and describe it as partly met.

Q62. So what indoor support does the system have today?

- **Places inside buildings:** the Florence Onny Auditorium (Graduate block, ground floor) and the Library (Admin block, first floor) are modelled with `insideBuilding` and `floor`. Routes end at the host building's entrance with an indoor instruction, e.g. "take the stairs to the First Floor".
- **Rooms in the database:** every building stores its rooms with floor and capacity (Room G6, the Great Hall, the Eva von Hirsch Auditorium and others).
- **Room search:** `/api/buildings/search?q=Room G6` returns Block G first, because room names are in the text index.

Q63. How would you extend it to full indoor navigation?

The same graph and the same Dijkstra, with more data: corridors, stairs and rooms become nodes with a floor number, and stairs or lifts become edges between floors. Nothing in the algorithm changes, so with floor plans it becomes a data task, not a redesign. For positioning indoors, where GPS fails, I would start with QR codes at entrances and later use Wi-Fi or Bluetooth beacons.

Q64. Why not draw the floor plans yourself?

Guessed floor plans would give wrong directions, which is worse than none. I saw this outdoors: estimated coordinates produced wrong turn instructions until I corrected them against real positions.

On the day

- Run `npm run build` then `npm start` **before** you walk in, on the network you'll actually use, confirm "MongoDB connected successfully", and open `http://localhost:5000`.
- Demo order: Campus tab → Get Directions → route and turn list → Walk Demo → route to the **Library** to show the indoor step ("take the stairs to the First Floor") → submit a ticket → admin sign-in to see it with the bot reply.
- Have `npm test` ready to show the tests passing, and MongoDB Atlas open to show the ticket saved live.
- If you don't know an answer, say what you do know and how you would find out. Never guess numbers.